

**Task 1:** Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

---

## **Screenshots:**

1. Program Source Code –

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char command[100];

    printf("Enter a Linux command: ");
    scanf("%99s", command);

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("Fork failed");
        return 1;
    }

    if (pid == 0)
    {
        // Child Process
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());

        printf("Executing command: %s\n", command);

        execlp(command, command, NULL);

        perror("exec failed");
        exit(1);
    }
    else
    {
        // Parent Process
        printf("\nParent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);

        wait(NULL);

        printf("\nChild process completed.\n");
    }

    return 0;
}

```

In the above screenshot, a C program is developed that accepts a Linux command from the user, creates a child process using the `fork()` system call, executes the entered command using the `exec()` system call, waits for the child process to complete using `wait()`, and displays the Process IDs (PIDs) of both the parent and child processes.

## 2. Compilation –

```
ashborn@DESKTOP-I4G5Q4N:~/OSSP$ nano src/practical_week2.c
ashborn@DESKTOP-I4G5Q4N:~/OSSP$ gcc src/practical_week2.c -o bin/practical_week2
ashborn@DESKTOP-I4G5Q4N:~/OSSP$ █
```

We have successfully compiled the C program using GCC compiler. The command `gcc src/practical_week2.c -o bin/practical_week2` compiles the source code and generates an executable file named `practical_week2`. The program is successful as there are no compilation errors shown.

## 3. Program Execution –

```
ashborn@DESKTOP-I4G5Q4N:~/OSSP$ ./bin/practical_week2
Enter a Linux command: ls

Parent Process
Child Process
Parent PID : 546
Child PID : 547
Child PID : 547
Parent PID : 546
Executing command: ls
LICENSE Makefile README.md assets bin docs include obj scripts src tests

Child process completed.
ashborn@DESKTOP-I4G5Q4N:~/OSSP$ █
```

We can see the execution of our program. The user enters the Linux command `ls`, after which the program creates a child process using `fork()`. The child process executes the command using `exec()`, while the parent process waits for the child to finish using `wait()`. The output displays the Parent PID, Child PID, the execution of the `ls` command, and the message showing that the child process is successful. This demonstrates the working of Linux process creation and command execution.

---

**Task-2:** Using Linux terminal commands (`uname`, `lscpu`, `ps`, `top`), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

# Task 2 REPORT:

## Objective:

To investigate the relationship between hardware resources and operating system services using Linux terminal commands such as `uname`, `lscpu`, `ps`, and `top`, and to understand how the operating system abstracts CPU, memory, storage, and I/O devices.

### 1. Introduction –

An operating system acts as an interface between the user, application programs, and computer hardware. It manages hardware resources such as the CPU, memory, storage devices, and input/output devices and provides convenient services to applications.

Linux provides several terminal commands that allow users to observe how these resources are managed. In this practical, the commands `uname`, `lscpu`, `ps`, and `top` are used to examine system information, CPU resources, running processes, and resource utilization. The `df` command and `/dev` directory are also useful for understanding storage and I/O device abstraction.

### 2. `uname` Command –

command: `uname -a`

```
ashborn@DESKTOP-I4G5Q4N:~$ uname -a
Linux DESKTOP-I4G5Q4N 4.4.0-19041-Microsoft #5794-Microsoft Mon Apr 07 17:55:00 PST 2025 x86_64 x86_64 x86_64 GNU/Linux
ashborn@DESKTOP-I4G5Q4N:~$
```

The `uname` command is used to display basic information about the operating system and the system on which it is running. The `-a` option displays all available system information, including the kernel name, hostname, kernel release, kernel version, machine architecture, and operating system.

This command helps identify the Linux environment and the operating system kernel that provides services to applications and manages the underlying hardware.

### 3. Lscpu Command –

command: lscpu

```
ashborn@DESKTOP-I4G5Q4N:~$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          36 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 4
On-line CPU(s) list:    0-3
Vendor ID:              GenuineIntel
Model name:             Intel(R) Core(TM) i3-7100U CPU @ 2.40GHz
CPU family:             6
Model:                 142
Thread(s) per core:     2
Core(s) per socket:     2
Socket(s):              1
Stepping:               9
CPU(s) scaling MHz:    100%
CPU max MHz:            2400.0000
CPU min MHz:            0.0000
BogoMIPS:               4800.00
Flags:                  fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2
                        ss36 nx lm sse4_1 sse4_2 movbe popcnt aes xsave osxsave avx f16c rdrand hypervisor lahf_lm abm
                        3dnowprefetch smap clflushopt ibrs ibpb stibp ssbd
Virtualization features:
Hypervisor vendor:      Windows Subsystem for Linux
Virtualization type:    container
ashborn@DESKTOP-I4G5Q4N:~$
```

The lscpu command displays detailed information about the CPU architecture and processor resources available to the operating system.

The output provides information such as the CPU architecture, number of CPUs, number of cores, number of threads per core, CPU model, and virtualization information.

The operating system uses this CPU information to manage processor resources and schedule processes for execution. Therefore, lscpu helps demonstrate the relationship between the physical CPU hardware and the services provided by the operating system.

### 4. ps Command –

command: ps -e

```
ashborn@DESKTOP-I4G5Q4N:~$ ps -e
  PID TTY          TIME CMD
    1 ?           00:00:00 init(Ubuntu)
    7 ?           00:00:00 init
   10 tty1       00:00:00 SessionLeader
   11 tty1       00:00:00 bash
   61 tty1       00:00:00 ps
```

The ps command displays information about processes currently running in the system. The -e option displays processes from all users.

Each process is associated with a Process ID (PID), which the operating system uses to identify and manage it. The output demonstrates how the operating system manages applications and system services as processes.

Thus, the ps command provides a view of how CPU resources are allocated to different processes by the operating system.

## 5. Top Command –

command: top

```
ashborn@DESKTOP-I4G5Q4N:~$ top
top - 06:09:24 up 6 min,  0 user,  load average: 0.52, 0.58, 0.59
Tasks:  5 total,   1 running,  4 sleeping,   0 stopped,   0 zombie
%Cpu(s):  1.1 us,  3.4 sy,   0.0 ni, 95.0 id,   0.0 wa,   0.5 hi,   0.0 si,   0.0 st
MiB Mem :  8073.4 total,  2384.3 free,  5689.1 used,   224.0 buff/cache
MiB Swap: 24576.0 total, 24369.3 free,   206.7 used.  2384.3 avail Mem
```

| PID | USER    | PR | NI | VIRT  | RES  | SHR  | S | %CPU | %MEM | TIME+   | COMMAND       |
|-----|---------|----|----|-------|------|------|---|------|------|---------|---------------|
| 1   | root    | 20 | 0  | 11244 | 1140 | 1052 | S | 0.0  | 0.0  | 0:00.09 | init(Ubuntu)  |
| 7   | root    | 20 | 0  | 11212 | 620  | 480  | S | 0.0  | 0.0  | 0:00.00 | init          |
| 10  | root    | 20 | 0  | 11260 | 576  | 524  | S | 0.0  | 0.0  | 0:00.01 | SessionLeader |
| 11  | ashborn | 20 | 0  | 14112 | 3844 | 3728 | S | 0.0  | 0.0  | 0:00.12 | bash          |
| 63  | ashborn | 20 | 0  | 17356 | 3880 | 1644 | R | 0.0  | 0.0  | 0:00.01 | top           |

The top command provides a real-time view of running processes and system resource utilization.

It displays information about CPU usage, memory usage, running and sleeping processes, system load average, process IDs, and the users running the processes.

This demonstrates how the operating system continuously monitors and manages system resources. The operating system uses this information to schedule processes and allocate CPU and memory resources efficiently.

## 6. Storage Management using df –

command: df -h

```
ashborn@DESKTOP-I4G5Q4N:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
rootfs          477G  137G  340G   29% /
none            477G  137G  340G   29% /dev
none            477G  137G  340G   29% /run
none            477G  137G  340G   29% /run/lock
none            477G  137G  340G   29% /run/shm
none            477G  137G  340G   29% /run/user
tmpfs           477G  137G  340G   29% /sys/fs/cgroup
C:\              477G  137G  340G   29% /mnt/c
D:\              85G   1.9G   83G    3% /mnt/d
E:\             342G   2.3G  340G    1% /mnt/e
F:\             395G   791M  394G    1% /mnt/f
ashborn@DESKTOP-I4G5Q4N:~$
```

The `df -h` command displays information about the available disk space on mounted filesystems. The `-h` option presents the values in a human-readable format.

The output shows the total storage capacity, used space, available space, and percentage of storage currently being used.

The operating system abstracts physical storage devices through filesystems, allowing users and applications to work with files and directories without directly managing physical disk sectors.

## 7. I/O Device Abstraction using `/dev` – command: `ls /dev | head`

```
ashborn@DESKTOP-I4G5Q4N:~$ ls /dev | head
block
fd
kmsg
lxss
null
ptmx
pts
random
shm
stderr
```

Linux provides access to many hardware and system devices through the /dev directory. Devices are represented using special device files, which provide an interface between applications and hardware.

This abstraction allows applications to interact with devices through standard operating system interfaces instead of directly communicating with the physical hardware. Therefore, the /dev directory demonstrates how Linux provides an abstraction layer for input/output devices.

## 8. OS Abstraction of Hardware Resources –

The operating system abstracts hardware resources by providing a convenient interface between applications and the physical hardware.

8.1 CPU Abstraction – The operating system manages the CPU by scheduling processes and allocating processor time to different tasks. Applications do not directly control the CPU. Instead, they request CPU resources through operating system services. The `lscpu` command provides information about the CPU resources available to the operating system, while `ps` and `top` show how processes are managed and executed.

8.2 Memory Abstraction – The operating system manages the computer's main memory and allocates memory to running processes. Applications can use memory without needing to directly manage the physical RAM locations. The `top` command provides information about memory usage and shows how memory resources are being utilized by running processes.

8.3 Storage Abstraction – The operating system provides filesystems, directories, and files as an abstraction over physical storage devices. Users and applications can access files without needing to know the physical location of the data on the disk. The `df -h` command demonstrates filesystem storage capacity, usage, and availability.

8.4 I/O Device Abstraction – Linux provides interfaces through which applications can interact with input/output devices. The /dev directory contains device files that represent various devices and system interfaces. The `ls /dev` command demonstrates this device abstraction. Applications can communicate with devices through operating-system-provided interfaces instead of directly controlling the hardware.

## 9. Relationship Between Hardware and OS Services –

The commands used in this practical demonstrate the relationship between hardware resources and operating system services.



| Hardware/Resource       | Linux command | What it demonstrates                        |
|-------------------------|---------------|---------------------------------------------|
| CPU                     | lscpu         | CPU architecture, cores and threads         |
| Processes/CPU usage     | ps, top       | Process management and resource utilization |
| Memory                  | top           | Memory allocation and usage                 |
| Storage                 | df -h         | Filesystem capacity and storage usage       |
| I/O devices             | ls /dev       | Device-file abstraction                     |
| Operating system/kernel | uname -a      | Linux kernel and system information         |

The operating system therefore acts as an abstraction layer between applications and physical hardware.

## Conclusion –

The practical demonstrated how Linux provides operating system services for managing hardware resources. The uname command provided information about the Linux system and kernel, while lscpu displayed CPU-related information. The ps and top commands demonstrated process management and CPU and memory utilization.

The df -h command demonstrated how Linux manages storage through filesystems, while the /dev directory demonstrated the abstraction of I/O devices through device files. Thus, the operating system hides the complexity of the underlying hardware and provides standardized interfaces that allow users and applications to efficiently use CPU, memory, storage, and I/O resources.